

```

// interop_comms.h

#include <Arduino.h>
#include <SoftwareSerial.h>

#include ".common/comm.h"

bool NodeMCUTransmit(Requests request, SoftwareSerial *serial, const char *payload,
char *out_buffer,
                        size_t buffer_size);

// interop_comms.cpp

#include <Arduino.h>
#include <SoftwareSerial.h>

#include ".common/comm.h"
#include "interop_comms.h"

bool NodeMCUTransmit(Requests request, SoftwareSerial *serial, const char *payload,
char *out_buffer,
                        size_t buffer_size)
{
    size_t idx = 0;
    switch (request)
    {
        case Requests::ActiveBoxes:
            serial->print(ReqActBox);
            break;
        case Requests::UpdateBoxes:
            serial->print(UpdBoxes);
            break;
        default:
            return false;
    }
    serial->print(MSG_TERMINATOR);
    serial->flush();
    unsigned long start = millis();
    while (millis() - start < 1000)
    {
        while (serial->available())
        {
            char c = serial->read();
            if (c == MSG_TERMINATOR)
            {
                out_buffer[idx] = '\0';
                goto RESPONSE_DONE;
            }
            if (idx < buffer_size - 1)
                out_buffer[idx++] = c;
            else
                idx = 0;
        }
    }
}

```

```
}
return false;
RESPONSE_DONE:
if (request == Requests::UpdateBoxes)
{
    if (strcmp(out_buffer, "OK") != 0)
    {
        d_SerialPrintln("NodeMCU did not ACK update");
        return false;
    }
    if (payload != NULL)
    {
        serial->print(payload);
        serial->print(MSG_TERMINATOR);
        serial->flush();
    }
    return true;
}
return true;
}
```